

UNIDAD 5 – SCM (Software Configuration Management)

Definición y propósito

Disciplina que **mantiene la integridad de los artefactos** (Productos, Procesos, Ambientes) a lo largo del ciclo de vida. Permite identificar, controlar y recuperar configuraciones. Proceso de soporte que beneficia a PM, desarrollo, mantenimiento, SQA y usuarios.

Problemas que resuelve: "en mi máquina funciona", entregables que tardan días, no saber en qué cliente está el bug, probar en testing y no dev, no saber qué usuarios tienen v1.0.

Ítem de Configuración (IC / SCI)

Elemento bajo control de gestión de configuración. Su gestión debe **aportar valor**.

Identificado por: nombre, versión, autor/revisores, fecha, ítems relacionados, última modificación, tipo.

Tipos que pueden ser IC: código fuente y ejecutable, documentación, scripts, BD, modelos, servers, especificaciones, componentes de terceros, materiales de testing, herramientas.

Tipo de IC	Persistencia	Ejemplos
Proceso	viven durante el proyecto	Plan de Riesgos, Plan Calidad, Plan Proyecto, Plan CM, Documentos de requerimientos, estimaciones, matriz de comunicación, condiciones de prueba
Producto	trascienden al proyecto	Requerimientos, Arquitectura, Manual de Usuario, Código fuente, Scripts BD, Modelos de datos, Plan de Integración, Casos de prueba, Estándares de codificación

Criterios para seleccionar un IC: usado por 2+ personas, se espera que cambie, crítico para el éxito, interdependiente con otros.

⚠ Trampas: condiciones de prueba = proceso; manual usuario = producto; componentes de terceros sí se gestionan; no todos los entregables son ICs.

Conceptos clave

- **Configuración:** conjunto de ICs compilados en un ejecutable consistente.
- **Línea base (Baseline):** configuración verificada y aprobada, punto de partida. Solo cambia por procedimientos formales (IEEE 610.12-1990). Un IC puede estar en varias; una config puede tener ICs de distintas versiones.
 - **Tipos:** functional, allocated (SRS), developmental, product.
- **Versión:** instancia funcionalmente distinta.
- **Variante:** funcionalmente igual, distinta a nivel no funcional (Linux vs Windows).
- **Release:** distribución fuera del entorno de desarrollo. Establece una baseline, pero **baseline no implica release**. Requiere versión. Puede asociarse a beta.

Semantic Versioning (MAJOR.MINOR.PATCH):

- MAJOR: cambios incompatibles.
- MINOR: funcionalidad nueva compatible.
- PATCH: bug fixes compatibles.
- Etiquetas: -beta, +build123.

Trazabilidad

Capacidad de rastrear un IC en todo el proceso.

- **Horizontal:** vincula elementos con el requerimiento que los generó, a través de distintos niveles del ciclo.
- **Vertical:** elementos del mismo tipo dentro del mismo nivel.

Proceso SCM – 6/7 actividades (SWEBOK / IEEE 828-2012)

1. Administración del proceso SCM

Etapa previa al proyecto. Entender contexto (JIRA, GitLab, GitHub) y construir el **Plan de SCM (SCMP)**.

El SCMP incluye: lista de ICs y momento de ingreso; estándares de nombres/versionado; estrategias de branching; reglas de la herramienta CM; procedimientos de build/release; momentos de auditoría.

Aspectos: organización y responsabilidades, recursos, selección de herramientas, vendor/subcontratista, interfaces.

Vigilancia: verifica que el SCMP se cumpla (mediciones, auditorías in-process).

2. Identificación de la configuración

Define qué será IC y características de la baseline/release. Actividades sobre código, ambientes y documentación.

Software Library: working (codificación), project support (testing), master (producto terminado).

3. Control de cambios

Objetivo: integridad ante cambios. Etapa más importante.

Pasos: 1) Identificar propósito; 2) Evaluar impacto; 3) Aprobar/rechazar (SCCB); 4) Planificar; 5) Implementar y verificar; 6) Documentar.

SCCB (Configuration Control Board): autoridad para aprobar/rechazar. Composición: program/systems/software engineering, SQA, SCM, PO, analista funcional, usuarios representativos, independent test.

- Tiene el **governance**; puede rechazar un cambio del PO; lo rechazado puede reevaluarse; cambios de emergencia se regularizan a posteriori; todo despliegue a producción se registra como cambio.

SCR (Software Change Request): pedido formal, registra tipo (defecto/enhancement) e impacto técnico.

Deviation (autorización **previa**) vs **Waiver** (autorización **posterior** para aceptar IC fuera de spec).

Métricas de cambios: Successful Changes, Changes in Backlog, Emergency Changes (idealmente bajo).

4. Status & Accounting

Registrar y reportar para que la trazabilidad funcione. Responde: **QUÉ, CUÁNDO, QUIÉN, ALCANCE, QUIÉN APROBÓ, QUIÉN SOLICITÓ.**

No resuelve: detectar código sin requerimiento (eso es auditoría física) ni ICs faltantes (eso es identificación).

Reportes: transaction log, change log, SCI delta, SCI status, changes in progress, change request status, change completion.

5. Auditoría

Actividad **independiente**, control posterior, no parte del proceso.

Tipo	Verifica
Funcional (FCA)	Que lo implementado cumple los requerimientos (testing, inspección, demostración)
Física (PCA)	Que el as-built coincide con la doc; detecta backdoors / código extra
De Proceso	Que se cumplió el proceso SCM (tickets, sprints, deploys)

⚠ Las auditorías físicas NO se reemplazan por testing (QC).

6. Release Management & Delivery (SW Building + Releasing)

Construcción exitosa del paquete y liberación controlada. Debe poder **reproducir releases previos**. Las herramientas de build también bajo SCM.

Consideraciones de release: quién recibe el cambio, despliegue, entornos, roll forward/back, validación, riesgos, aprobaciones.

7. SCM Tools

Categoría	Para
Individual	Org. chicas, sin variantes
Project-related	Medianas/grandes con desarrollo paralelo
Companywide-process	Procesos formales con certificación

Virtualización: Metal (sin overhead, sin snapshots) · VM (snapshots, pérdida grande de performance) · Contenedores (1 servicio por container, casi sin overhead, muy rápido).

Branching

Estrategia	Concepto
Gitflow	master + develop + feature/release/hotfix
GitHub Flow	main + ramas por feature vía pull request
Trunk-Based	Todos en main, integración frecuente, feature toggles

Ambientes

Desarrollo (local) · Testing (QA) · Staging (simula prod con datos reales) · Producción (usuarios finales).

Builds (deben ser automáticos)

Tipo	Cuándo
Local	dev en su entorno, unit tests

Integration	entorno completo para tests de integración
Nightly	diario, reportes de estabilidad
Release	crear nueva versión para usuario final

⚠ "Server builds" y "Weekly builds" no son tipos según la cátedra.

CI / CD / Deploy Continuo

- CI (Integración Continua): build automático y testeable al menos 1×/día. Prácticas: repo único, build automatizado, build testeable, todo commit construido, build rápido en clon de producción, transparencia, despliegue automatizado.
 - **Responsabilidades del equipo:** check-in frecuente, no subir código roto/no testeado, no irse hasta que el build compile.
- **Continuous Delivery:** automatiza hasta staging; producción requiere **aprobación manual**.
- **Continuous Deployment:** automatiza incluso producción. **Requiere CD previo**.
- **Pipeline:** build → testing → deployment. Etapas: Testing/QA → Governance/Approvals → Deployment → Verification → Rollback.

Estrategias de Deployment

Estrategia	Concepto
Basic	Todos los nodos a la vez. Simple, riesgoso.
Blue/Green	Dos ambientes paralelos, se conmuta. Costoso (doble infra).
Rolling	Nodos gradualmente. App/BD deben tolerar +1/-1 simultáneos.
Canary	Fases (2/10/50/75/100%). Complejo de monitorear.

DevOps

Wall of Confusion: Dev quiere cambio, Ops quiere estabilidad. DevOps reduce el tiempo entre cambio y producción, garantizando calidad. **No es metodología** — son prácticas + herramientas + cultura.

CALMS: Cultura · Automatización · Lean · Métricas · Sharing.

Ciclo: Plan → Discover → Build → Test (dev) → Deploy → Operate → Observe (ops), con Continuous

Feedback.

Prácticas por etapa: planificación (colaboración) · coding/building (pipelines, IaC) · testing (automated, chaos/fault injection) · release (CI/CD) · operación (containers, monitoring/alerting).

Métricas DORA: Deployment Frequency · Lead Time for Changes · Change Failure Rate · MTTR.

UNIDAD 6 – SQA (Software Quality Assurance) y Testing

El paraguas SQA

Rama	Naturaleza	Sobre qué
QA (Quality Assurance / Peer Review)	Estática	Código, diseño, requerimientos, casos
QC (Testing)	Dinámica (ejecuta el SW)	Unit, integración, sistema, regresión, UAT, UI/UX

SQA es preventivo, orientado al proceso, todo el ciclo. QC es correctivo, sobre el producto. La calidad no se "inyecta" al final.

Verificación vs Validación

- **Verificación:** ¿lo construyo correctamente? (unit, integración, regresión)
- **Validación:** ¿construyo el producto correcto? (UAT, UI/UX)

Tipo	Verif.	Valid.
Unitarias, Integración	☑	
Regresión, Sistema	☑	☑
UAT, UI/UX		☑

Definiciones (IEEE)

Ejecutar un sistema bajo condiciones específicas, observar resultados y evaluar.

Aclaraciones críticas:

- Las pruebas encuentran **fallas**, no defectos. **Solo muestran presencia, nunca ausencia.**
- Se parte del supuesto de que **hay** defectos.
- Si buscás "no errores", el sesgo de confirmación arruina la prueba.

Objetivos: encontrar fallas; eficiente (rápido/barato); eficaz (fallas reales, priorizando críticas).

Por qué se producen fallas: requerimientos poco claros/cambiantes; diseño complejo; falta de experiencia/tiempo/conocimiento; problemas de entorno.

Cadena Equivocación → Defecto → Falla

Concepto	Definición	Cardinalidad
Equivocación	Acción humana incorrecta	1 → N defectos
Defecto	Paso/dato incorrecto, ausencia de característica	1 → 0..N fallas
Falla	Resultado de ejecución incorrecto (manifestación del defecto)	1 → 1..N defectos

- **Incidente:** ocurrencia que requiere investigación. No todo incidente es falla.
- **Falso positivo:** se reporta falla, pero funcionaba.
- **Depuración:** eliminar el defecto. **No es tarea de prueba;** es consecuencia. Puede introducir nuevos defectos.

Pasos de depuración: 1) Detectar (falla → defecto) 2) Depurar (razón, solución); 3) Volver a probar (regresión); 4) Aprender.

Economía del testing

Seguir probando es **decisión económica**. **Cuándo parar:** pasa el set diseñado · "Good Enough" ·

fallas detectadas $\approx$ estimadas.

Abaratamiento: diseñar para testeabilidad (modular, ocultamiento, puntos de control).

Enfoques

Caja Negra (funcional)

Prueba lo que el SW **debería hacer**. Ignora estructura interna. Dividir entradas en **clases de equivalencia**.

Caja Blanca (estructural)

El tester conoce código/diseño/implementación. Aumenta cobertura.

Grados de cobertura (de menor a mayor):

1. **Sentencias** — cada instrucción.
2. **Decisiones** — cada salida IF/WHILE (incluye sentencias).
3. **Condiciones** — cada expresión lógica (A AND B); incluye decisiones.
4. **Camino básico** — todos los caminos independientes. La más completa.

100% decisiones $\Rightarrow$ 100% sentencias. Condiciones $\Rightarrow$ decisiones.

Camino básico: grafo de flujo (nodos = sentencias, aristas = transiciones).

Complejidad Ciclomática (McCabe): caminos independientes a testear.

- $V(G) = \text{Aristas} - \text{Nodos} + 2$ = número de regiones del grafo.
- **Mínimo** = 1. "Mayor que 10" es guía, no regla.
- Mayor CC $\Rightarrow$ menor mantenibilidad.
- Es métrica de **producto**, no de recursos.
- Determina cantidad de casos en caja blanca.

SonarQube: determina **deuda técnica** y esfuerzo para pagarla. **No** ejecuta testing.

Caja Gris

Caja negra + parte de la implementación.

Técnicas de diseño (Caja Negra)

Partición de equivalencia

Por condición de entrada:

Tipo	Válidas	Inválidas
Rango (ej. 100		

Si una clase no se trata uniformemente, subdividir.

Condición de borde

Explora límites de clases existentes (no genera nuevas). Probar extremos y vecinos (umbral 10.000 $\rightarrow$ 9.999, 10.001). En conjuntos ordenados: primero, último, vacío.

Conjetura de errores (Prueba de sospechas)

Listar errores propensos según experiencia. Intuitiva, ad hoc. **Aplicar cuando:** componente apurado, modificado por varios, anidaciones complejas, copy & paste. El programador es la mejor fuente.

Clases inválidas de distinto tipo

Numéricos vs alfabéticos, fechas erróneas, **condiciones cruzadas** entre campos (ej. CUIT contiene DNI; divorciado ignora datos del cónyuge).

Niveles de prueba

Nivel	Descripción
Unitario	Unidad clara, por quien la construyó, sobre diseño detallado. Módulos cohesivos son más fáciles.
Integración	Partes juntas. No Incremental (Big Bang) / Incremental (Bottom-Up, Top-Down, Sandwich).
UAT (Aceptación)	Usuario verifica requerimientos. Caja negra pura.
Humo	Verifica rápido que el sistema básico no está roto.
Alfa	Usuario en instalaciones del desarrollador.
Beta	Usuario en sus propias instalaciones.

Pruebas no funcionales

Prueba	Qué verifica
Volumen	Volumen máximo (almacenamiento, procesamiento, transmisión)
Performance	Tiempos de respuesta con distintos niveles de carga
Estrés	Excede capacidad. Encuentra punto de ruptura
Seguridad / Pen Testing	Vulnerabilidades explotables
Regresión	Cambio no rompe lo existente. Ideal automatizar

Prueba estática vs dinámica

- Dinámica: ejecuta el SW.
- Estática: no ejecuta. Peer reviews, análisis (SonarQube), inspecciones, walkthroughs.

Modelos de ciclos

- **V-Model:** cada fase de definición tiene su fase de verificación. User Requirements ↔ Acceptance · Software Spec ↔ System · High Level Design ↔ Integration · Low Level Design ↔ Unit · Coding (vértice).
- **Pirámide:** base = unit (más, baratos, rápidos), medio = integración, cima = E2E/UI (menos, caros).

- Cuadrantes (Agile):
 - Q1 — Tecn./Soportan equipo (auto): Unit, Component, Integration.
 - Q2 — Negocio/Soportan equipo (auto+manual): Functional, Story, Prototypes, Simulations.
 - Q3 — Negocio/Miran producto (manual): Exploratory, Usability, UAT, Alpha/Beta.
 - Q4 — Tecn./Miran producto (semi-auto): Performance, Volumen, Security, "ilities".

Agile Testing — TDD

Ciclo: Rojo (test falla) → Verde (código mínimo que pasa) → Refactor.

Ventajas: modular, mantenible, refactoring de bajo riesgo, regresión cubierta, mejor cobertura.

Desventajas: errores compartidos código/test, funciones difíciles de testear, mantenimiento de tests, falso nivel de confianza.

Enfoques: Script-Driven vs Exploratory

Script-Driven (Guionado)

Diseño antes de ejecución. Controlado, predecible. Costoso, rígido.

Exploratory Testing (ET) — *James Bach*

Aprendizaje + diseño + ejecución simultáneos. El tester ajusta a medida que ejecuta.

- **Etapas:** reconocimiento → diseño → ejecución → interpretación.
- **Ventaja:** flexible, cubre situaciones inesperadas.
- **Desventaja:** difícil reproducir, dependiente del tester.
- **Habilidades clave:** diseño de tests, observación cuidadosa (observación ≠ inferencia), pensamiento crítico, ideas diversas (heurísticas), recursos ricos.

Charter: misión de una sesión (tester o lead).

Freestyle ET: sin script, output = bug reports.

Session-Based Test Management: notas escritas + entrevista diaria del lead → auditable y medible.

Cuándo usar ET: feedback rápido, aprender producto, diversificar scripted, encontrar el bug más importante, verificar a otro tester, investigar un defecto.

Gestión: Delegación (lead asigna charters) · Participación (lead testea junto al equipo). Pair testing ("driver + observers") produce más ideas.

Oráculos — Heurística HICCUPP

Oráculo: principio para reconocer un problema. Producto debe ser consistente en su:

Letra	Criterio
H	History — versiones previas, claims previos
I	Image — imagen de empresa
C	Comparable Product
C	Claims — reqs, marketing, help
U	User Expectations
P	Product — coherencia interna
P	Purpose — propósito del SW (el más fuerte)

Efectividad:

1. Claims, Comparable Product, History ☒ más efectivos para que se corrija.
2. Product, Purpose ☒ influyen en devs, no en managers.
3. Image, User Expectation ☒ "frosting", débiles solos.

⚠ HICCUPP no reemplaza reglas de negocio para cálculos complejos.

Peer Reviews — *Karl Wieggers*

Actividad estática. De más formal a menos: Inspection ☒ Team Review ☒ Walkthrough ☒ Pair Programming ☒ Peer Deskcheck / Passaround ☒ Ad Hoc.

Tipo	Características
Inspection	Más sistemática. 7 etapas (planning, overview, preparation, meeting, rework,

	follow-up, causal analysis). Roles: moderator, reader, recorder , autor. Detecta 16-20 defectos/KLOC vs 3/KLOC informal.
Team Review	"Inspection-lite". Sin reader. Autor puede liderar. ~2/3 de efectividad.
Walkthrough	Informal. Autor dominante. Sin roles ni métricas. Útil en mantenimiento.
Pair Programming	2 devs en 1 estación. Revisión continua. Promueve collective ownership.
Peer Deskcheck	1 solo revisor. El más barato. Depende del revisor.
Passaround	Peer deskcheck en paralelo con varios revisores. Mitiga riesgo del único revisor. Sin sinergia grupal.
Ad Hoc	Spur-of-the-moment. Sin impacto sistémico.

Actividades incluidas

Tipo	Plan	Prep	Meeting	Correct	Verif
Inspection	☒	☒	☒	☒	☒
Team Review	☒	☒	☒	☒	—
Walkthrough	☒	—	☒	☒	—
Pair Programming	☒	—	continuo	☒	☒
Deskcheck/Passaround	—	☒	posible	☒	—
Ad Hoc	—	—	☒	☒	—

Elección por riesgo

Alto ☒ Inspection · Bajo ☒ más baratas.

Factores de riesgo: tecnologías nuevas, lógica/algoritmos complejos, presión de tiempo, devs sin experiencia, componentes safety-critical o reutilizables/template, muchos modos de falla, múltiples interfaces.

Método por objetivo

Objetivo	Método principal
Conformidad con specs	Inspection, Team Review, Deskcheck
Conformidad con estándares	Inspection, Deskcheck
Compleitud/correctitud	Inspection, Walkthrough

Componentes críticos	Solo Inspection
Mejora de proceso	Inspection, Team Review
Medir calidad de documento	Solo Inspection

Clasificación de pruebas

- **Conocimiento:** caja blanca/negra/gris.
- **Ejecución:** automatizadas/manuales/semi-automatizadas.
- **Tipo:** funcionales/no funcionales.
- **Nivel:** unitarias/integración/sistema/aceptación.

Principios básicos

- Resultado esperado es parte integrante de la prueba.
- Programador no prueba su propio componente.
- Revisar a fondo el resultado.
- No suponer que no hay errores.
- Probar es desafío intelectual.
- **Objetivo:** encontrar fallas, no demostrar que no hay.

Conclusiones

- Las pruebas no mejoran el SW, solo muestran fallas.
- Buen diseño beneficia prueba, corrección y mantenimiento.
- No probar no elimina errores ni abarata.
- **Lo más barato es no introducir defectos.**

UNIDAD 7 / PARTE 3 – Métricas de Software

Conceptos

Concepto	Definición	Ejemplo
Medida	Indicación cuantitativa de un atributo	"10.000 LDC"

Métrica	Medida del grado en que un sistema posee un atributo (no cualquier dato numérico)	"500 LDC/persona-mes"
Indicador (KPI)	Métrica o combinación que da visión más profunda; para toma de decisiones	productividad + defectos/KLOC

Componentes de un KPI: objetivo, interpretación, método (fuente/fórmula/visualización), frecuencia, valores esperados/rangos, representación.

GQM (Goal / Question / Metric)

1. Goal (conceptual): SMART. Ej. "Mejorar calidad de código 5% en un año".
2. Question (operacional): ¿detecto muchas fallas? ¿el usuario está conforme?
3. Metric (cuantitativo): N° fallas, % cobertura, satisfacción, deuda técnica.

Pasos del programa de métricas

1. Identificar objetivos (G).
2. Hallar preguntas (Q).
3. Definir métricas (M).
4. Datos históricos.
5. Proceso de recolección.
6. Validar y analizar.
7. Usar en decisiones.

Tipos de métricas

Familia	Mide	Ejemplos
Proceso	Actividades de desarrollo/mantenimiento	Lead/Cycle/Touch Time (Kanban), Earned Value, % desvío, defectos por fase, duración promedio
Producto	Artefactos/entregables	LOC, Function Points, Use Case Points, Complejidad Ciclomática , Cohesión/Acoplamiento, ISO 25000, Deuda Técnica , MTBF, MTTR, fallas
Recursos	Entidades que ejecutan el proceso	Fallas/tester, LOC/dev, esfuerzo de codificación, edad/experiencia/costo por rol, cantidad de proyectos

Earned Value

Métrica	Fórmula	Esperado
Schedule Variance (SV)	EV - PV	≥ 0
SPI (Schedule Perf. Index)	EV / PV	≥ 1

Cost Variance (CV)	EV - AC	≥ 0
CPI (Cost Perf. Index)	EV / AC	≥ 1

(EV = valor ganado, PV = valor planeado, AC = costo real)

Métricas de Testing

- Productividad de diseño: condiciones/tiempo.
- Productividad de ejecución: casos/tiempo.
- Eficacia pre-release: incidentes aceptados / total reportados $\times 100$.
- Eficacia post-release: fallas reportadas / (fallas + fallas por usuario) $\times 100$.
- Release readiness: índice de severidad por defectos abiertos.

Recomendaciones

- Pocos indicadores simples, según objetivos. No medir todo de entrada.
- No buscar resultados a corto plazo. Sostener en el tiempo. **Automatizar** captura.
- **No medir personas ni evaluar por métricas** (distorsiona).
- Las métricas son datos objetivos para **toma de decisiones**.

"You can't control what you can't measure" — Tom DeMarco

RESUMEN DE TRAMPAS FRECUENTES

Afirmación trampa	Verdad
Un baseline siempre genera un release	✗ Release implica baseline, no al revés
Auditorías físicas se reemplazan por testing	✗ Son independientes
Componentes de terceros no se gestionan	✗ Sí deben incluirse
Para deploy continuo no hace falta delivery	✗ Deploy continuo requiere delivery previo
Métrica = cualquier dato numérico	✗ Evalúa qué tan presente está un atributo
El testing demuestra ausencia de defectos	✗ Solo presencia

Complejidad Ciclomática mínima = 0	✗ Mínimo = 1
CC > 10 no permitido	✗ Es guía, no regla
CC es métrica de recursos	✗ De producto
SonarQube ejecuta testing	✗ Determina deuda técnica
Status & Accounting gestiona trazabilidad sola	✗ La soporta, no la gestiona sola
SCCB ejecuta auditorías	✗ gobierna control de cambios
HICCUPP reemplaza reglas de negocio	✗ No, para cálculos complejos
Prueba estática y dinámica son lo mismo	✗ Estática no ejecuta
Passaround = Peer Deskcheck	Es Deskcheck múltiple y concurrente
Depuración es parte del testing	✗ Es consecuencia
SPI = EV - PV	SPI = EV / PV; SV = EV - PV
Walkthroughs ≈ Inspecciones	✗ Inspecciones detectan más defectos/hora